

Backend Architecture Handoff (Spring Boot + JWT)

1. Tech Stack

- Java 17
- Spring Boot
- Spring Security (JWT)
- Spring Data JPA (Hibernate)
- MySQL
- Maven

2. Package Structure

Root package: `com.example.demo`

- `config` -> Spring config (`SecurityConfig`)
- `security` -> JWT filter, JWT service, user details service
- `controller` -> REST API layer
- `service` -> business logic
- `repository` -> DB query layer (JPA interfaces)
- `model` -> JPA entities / table mappings
- `dto` -> request/response contracts
- `exception` -> global error handling
- `util` -> helper functions

3. Architecture Flow

Client request flow:

1. Request enters `SecurityFilterChain`
2. `JwtAuthenticationFilter` validates bearer token
3. Controller receives request
4. Controller calls service
5. Service calls repository
6. Repository executes SQL via JPA/Hibernate
7. Service maps entity to DTO
8. Response returned as JSON

4. Security Architecture

Public APIs (no token)

- `/auth/login`
- `/auth/register`
- `/auth/google`
- `/auth/apple`
- `/auth/check-email`
- `/auth/check-username`
- `/auth/forgot-password`
- `/auth/reset-password`
- `/auth/verify`

Protected APIs

- All remaining APIs require `Authorization: Bearer <token>`

Role Authorization (method-level)

- `DOCTOR` -> diagnosis, prescription, doctor dashboard, billing access
- `NURSE` -> vitals, queue, nurse dashboard, nurse notes, tasks
- `PATIENT` -> read-only profile/history/diagnosis/prescription/vitals

Security implementation files:

- `config/SecurityConfig.java`
- `security/JwtAuthenticationFilter.java`
- `security/JwtService.java`
- `security/AppUserDetailsService.java`

5. Module Ownership

Auth

- Controller: `AuthController`
- Service: `AuthService`
- Repository: `UserRepository`
- Model: `User`

Patient

- Controller: `PatientController`
- Service: `PatientService`
- Repository: `PatientRepository`
- Model: `Patient`

Diagnosis

- Controller: `DiagnosisController`
- Service: `DiagnosisService`
- Repository: `DiagnosisRepository`
- Model: `Diagnosis`

Prescription

- Controller: `PrescriptionController`
- Service: `PrescriptionService`
- Repository: `PrescriptionRepository`, `MedicineRepository`
- Model: `Prescription`, `Medicine`

Vitals

- Controller: `VitalsController`
- Service: `VitalsService`
- Repository: `VitalsRepository`
- Model: `Vitals`

Queue

- Controller: `QueueController`
- Service: `QueueService`
- Repository: `QueueRepository`
- Model: `Queue`, `QueueStatus`

Doctor Dashboard

- Controller: `DoctorController`
- Service: `DoctorDashboardService`

Nurse Dashboard + Notes

- Controller: `NurseController`
- Service: `NurseService`
- Repository: `NurseNoteRepository`, `QueueRepository`, `VitalsRepository`, `PatientRepository`
- Model: `NurseNote`

Tasks

- Controller: `TaskController`
- Service: `TaskService`
- Repository: `TaskItemRepository`
- Model: `TaskItem`

Billing + Insurance

- Controllers: `BillingController`, `InsuranceController`
- Service: `BillingService`
- Repositories: `InvoiceRepository`, `InvoiceItemRepository`, `InsuranceProviderRepository`, `PatientRepository`
- Models: `Invoice`, `InvoiceItem`, `InsuranceProvider`

6. Database Model (High-Level Relations)

- `Patient` 1 -> many `Diagnosis`
- `Patient` 1 -> many `Prescription`
- `Prescription` 1 -> many `Medicine`
- `Patient` 1 -> many `Vitals`
- `Patient` 1 -> many `Queue`
- `Patient` 1 -> many `Invoice`
- `Invoice` 1 -> many `InvoiceItem`
- `Patient` 1 -> many `NurseNote`

7. API Design Notes

- Validation uses Jakarta validation annotations in DTOs (`@NotBlank`, `@NotNull`, `@Size`, etc.)
- Most create/update endpoints return `200 OK` + JSON payload
- PDF endpoints return binary file bytes:
 - `GET /prescriptions/{id}/pdf`
 - `GET /billing/invoices/{id}/pdf`
- Backward compatibility path exists:
 - diagnosis endpoints support both `/diagnosis` and `/diagnoses`

8. Error Handling Contract

Global exception handler: `exception/GlobalExceptionHandler.java`

Standard error payload:

```
```json
{
 "timestamp": "2026-05-03T11:30:00",
 "status": 400,
 "error": "Bad Request",
 "message": "Validation failed",
 "details": [
 "fieldName: reason"
]
}
```
```

9. Environment + Deployment Notes

- Keep JWT secret in environment variable/property (`app.jwt.secret`)
- Use profile-specific DB config for production (`application-prod.properties`)
- Ensure app port and EC2 security group inbound rules are aligned
- For protected APIs, frontend must attach bearer token on every request

10. Handoff Checklist for Backend Developer

1. Pull latest code and run `mvn clean install`
2. Validate DB schema and migrations
3. Confirm JWT login and role-based access
4. Run API smoke tests for doctor and patient flows
5. Verify dashboard endpoints and billing item lifecycle
6. Verify PDF endpoints return downloadable bytes